

Robotic Arm Project Workflow

3-DOF Desktop Pick-and-Place Arm + 1-DOF Gripper | 3-4 weeks at approximately 3 hours per day (63-84 hours)

1. Project Overview

Project objective

Design, simulate, manufacture, program, and demonstrate a compact robotic arm that can pick up a lightweight object from a known location and place it into one of two predefined bins. The project is deliberately limited to three positioning joints plus a gripper so that the mechanical design, ROS integration, and physical build can all be completed within the available timeframe.

Degrees of freedom

- Joint 1 - Base yaw: rotates the entire arm left and right.
- Joint 2 - Shoulder pitch: raises and lowers the upper arm.
- Joint 3 - Elbow pitch: extends or retracts the forearm.
- Gripper actuator: opens and closes the jaws; this is normally described separately from the arm's three positioning DOF.

The correct project description is therefore: 3-DOF arm plus a 1-DOF gripper, or four actuated joints in total. The arm can reach an x, y, z position, but it cannot independently control every aspect of end-effector orientation like a 6-DOF industrial arm.

Target specifications

- Maximum reach: approximately 250-350 mm.
- Payload: approximately 100-200 g, using foam blocks or similarly light objects.
- Structure: 3D-printed, laser-cut, or simple machined links with standard fasteners and bearings.
- Motion: slow, position-controlled, and limited to a safe tabletop workspace.
- Actuation: three position-controlled arm actuators plus one smaller gripper actuator.
- Feedback: measured joint position where available; commanded position may be used as an estimate only for actuators without feedback.
- Control computer: Ubuntu 24.04 with ROS 2 Jazzy.
- Simulation: RViz and Gazebo.
- Primary programming language: Python; C++ is an optional stretch goal.

Final demonstration

The final system should accept either a predefined target or a simple x, y, z coordinate, calculate the required joint angles, move through a safe approach path, close the gripper, lift the object, move to a bin, release it, and return to the home position.

2. System Architecture

Mechanical system

- Rigid table-mounted base with a rotating platform.
- Separate bearing support at the base so the actuator shaft does not carry the full radial and overturning load.
- Shoulder and elbow links designed around defined joint axes and realistic fastener access.
- Lightweight forearm and gripper to reduce shoulder torque.
- Mechanical hard stops or geometry that prevents damaging over-rotation.
- Cable routing that does not snag throughout the intended workspace.
- Optional extension spring or elastic counterbalance at the shoulder.

Actuation and electronics

- Prefer serial smart servos with position feedback when budget permits.
- If ordinary hobby servos are used, keep payload and reach conservative and clearly distinguish commanded position from measured position.
- Use a microcontroller as the low-level actuator interface if the actuators cannot connect directly to the ROS computer.
- Provide a separate, correctly rated actuator power supply; do not power the motors from the computer's USB port.
- Include a physical power-disconnect or emergency-stop method.
- Limit software speed, acceleration, and joint range before the first powered test.

ROS 2 package structure

- `mini_arm_description` - URDF/Xacro files, CAD meshes, joint limits, collision geometry, and RViz configuration.
- `mini_arm_bringup` - launch files, controller configuration, parameters, and simulation/hardware selection.
- `mini_arm_driver` - actuator communication, command output, measured joint-state input, timeouts, and hardware status.
- `mini_arm_kinematics` - forward kinematics, inverse kinematics, reachability checks, and coordinate transforms.

- mini_arm_tasks - homing, predefined poses, pick-and-place sequences, and recovery behavior.

Command and feedback flow

- A user or task node provides a target position or named pose.
- The kinematics node verifies that the target is reachable and calculates base, shoulder, and elbow angles.
- A trajectory is generated from the current joint state to the target joint state.
- The controller or driver sends position commands to the physical actuators.
- Actuator feedback is published as sensor_msgs/JointState.
- robot_state_publisher uses the URDF and joint states to publish the link transforms.
- RViz displays the measured or estimated physical configuration of the arm.

3. Detailed Four-Week Workflow

Week 1 - Requirements, calculations, and SolidWorks design

Goal: finish a mechanically credible design before purchasing or manufacturing final components.

Days 1-2: Requirements and workspace

- Write down maximum reach, payload, object size, desired pickup height, bin locations, and available table area.
- Choose conservative joint limits for the base, shoulder, and elbow.
- Sketch the desired side-view workspace and identify unreachable or unsafe regions.
- Decide whether the arm will be bolted to a base plate or clamped to a table.
- Define a home pose that is stable, compact, and away from hard stops.

Days 2-3: Torque and actuator sizing

- Create a simple mass estimate for the payload, gripper, forearm, elbow hardware, and upper arm.
- Calculate worst-case static torque with the arm horizontal: torque is approximately gravity multiplied by the sum of each mass times its distance from the joint.
- Apply a safety factor of approximately 1.5-2 to the static torque estimate.
- Size the shoulder actuator first, because it carries the largest combined load.
- Repeat the calculation for the elbow and base.
- Reduce link length or moving mass if the required torque becomes excessive.
- Do not finalize motor mounts until the actuator choice and mounting dimensions are confirmed.

Days 3-5: Parametric CAD skeleton

- Create a master layout sketch containing the base axis, shoulder axis, elbow axis, tool point, and nominal link lengths.
- Use global variables or equations for link length, joint spacing, bearing size, wall thickness, and fastener diameter.
- Place joint coordinate systems at the intended rotation axes; these will later simplify URDF creation.
- Create simplified actuator and bearing envelope models before designing detailed mounts.
- Test several arm configurations in the assembly and confirm the intended workspace.

Days 5-7: Detailed mechanical design

- Design the base, upper arm, forearm, actuator brackets, bearing supports, and gripper.
- Use ribs, fillets, shells, and material removal to improve stiffness-to-mass ratio.
- Provide tool access for every nut, screw, and bearing.
- Check for interference across the full joint range.
- Use SolidWorks mass properties to update the torque calculation.
- Create an exploded assembly and at least one manufacturing drawing.
- Freeze the first manufacturable revision at the end of the week.

Week 2 - URDF, visualization, simulation, and kinematics

Goal: make the exact CAD-based robot move correctly in software before assembling hardware.

Days 8-9: ROS workspace and robot description

- Create a Git repository and ROS 2 workspace.
- Create the `mini_arm_description` and `mini_arm_bringup` packages.
- Export simplified visual meshes from SolidWorks with consistent units and coordinate orientation.
- Use primitive boxes and cylinders for collision geometry wherever possible.
- Write the URDF as Xacro with links for the base, upper arm, forearm, and gripper.
- Define revolute joints, axes, position limits, velocity limits, and effort limits.
- Add inertial values using SolidWorks mass and center-of-mass information.

Days 10-11: RViz and TF validation

- Launch `robot_state_publisher` and `joint_state_publisher_gui`.
- Move each joint separately and confirm that it rotates about the correct axis.
- Inspect the TF tree and verify that link names and parent-child relationships are correct.
- Check that the visual meshes align with the joint frames.
- Correct scale errors, flipped axes, and mesh origins before proceeding.
- Save an RViz configuration showing the robot model, TF frames, and joint state.

Days 12-14: Simulation and kinematics

- Spawn the robot in Gazebo and verify that it remains stable under gravity.
- Configure position interfaces and a joint trajectory controller.
- Test slow single-joint commands before commanding multiple joints.
- Write a forward-kinematics function that calculates the tool position from the three joint angles.
- Compare the calculated tool position with the TF transform reported by ROS.
- Write an analytic inverse-kinematics solution using base angle plus two-link planar geometry.
- Reject targets outside the reachable workspace and handle both elbow-up and elbow-down solutions.
- Command several safe points in simulation and log the joint states.

Week 3 - Manufacturing, assembly, and hardware control

Goal: produce a slowly moving physical arm with reliable joint commands and state reporting.

Days 15-17: Manufacture and assemble

- Manufacture the frozen CAD revision and inspect critical dimensions before full assembly.
- Install bearings and shafts without forcing misaligned parts together.
- Assemble one joint at a time and check friction by hand before installing the actuator.
- Route cables with enough service loop for motion but not enough slack to snag.
- Verify the center of mass and confirm that the unpowered arm cannot fall dangerously.
- Update the CAD assembly to record any fit corrections made during construction.

Days 18-19: Low-level actuator tests

- Test each actuator individually at low speed and with the link disconnected when possible.
- Confirm direction, zero position, allowable range, current behavior, and feedback.
- Set conservative firmware or actuator limits.
- Create a repeatable mechanical or sensor-based homing procedure.
- Test the emergency-stop or power-disconnect method.
- Do not test coordinated motion until every individual joint behaves correctly.

Days 20-21: ROS hardware driver

- Create `mini_arm_driver` and establish communication with the actuators or microcontroller.
- Subscribe to safe joint-position or trajectory commands.
- Publish measured joint positions, velocities when available, and timestamps.
- Implement a command timeout that stops or holds motion if communication is lost.
- Add parameters for joint IDs, directions, zero offsets, and limits.

- Compare physical joint positions with the RViz model.
- Calibrate zero offsets so the physical arm and URDF agree.

Week 4 - Task automation, testing, and documentation

Goal: turn individual joint control into a repeatable pick-and-place demonstration.

Days 22-24: Safe poses and motion sequencing

- Define named poses for home, pickup approach, pickup, lift, bin approach, bin drop, and recovery.
- Move between poses at reduced speed while watching for cable and collision problems.
- Approach objects from above or from one consistent direction.
- Always lift the object before rotating the base toward the bin.
- Add pauses or completion checks before closing and opening the gripper.
- Return to the home pose after a successful or aborted task.

Days 25-26: Pick-and-place task node

- Create mini_arm_tasks with a simple state machine or ROS action.
- Accept a named pickup location, named bin, or validated x, y, z target.
- Check inverse-kinematics success before starting motion.
- Report the active step and any failure reason.
- Allow cancellation and transition to a safe recovery pose.
- Run at least ten repeated simulated cycles before testing the complete physical sequence.

Days 27-28: Validation and project handoff

- Complete repeated physical pick-and-place trials with lightweight objects.
- Record success rate, positioning error, cycle time, and observed failure modes.
- Capture screenshots of the CAD assembly, RViz model, TF tree, and Gazebo simulation.
- Record a short video showing the physical arm and the corresponding RViz state.
- Write setup, launch, calibration, and operating instructions.
- Document limitations and identify the next mechanical and software improvements.

4. Core Engineering and Coding Tasks

Static torque calculation

For each gravity-loaded joint, estimate the worst-case horizontal holding torque using: $\text{torque} = g \times (\text{payload mass} \times \text{payload distance} + \text{sum of each link/component mass} \times \text{its center-of-mass distance})$. Recalculate after obtaining SolidWorks mass properties. The actuator's practical

continuous capability, rather than only its advertised stall torque, should exceed the required holding torque with an appropriate safety factor.

Three-joint inverse kinematics

- Calculate base yaw with $\text{atan2}(y, x)$.
- Convert the horizontal target into radial distance from the base axis.
- Solve the shoulder and elbow as a two-link planar arm using the radial distance and target height.
- Clamp numerical values carefully before inverse cosine calculations.
- Reject targets that are too close, too far away, below the table, or outside joint limits.
- Choose one consistent elbow configuration unless there is a reason to switch.
- Compare the inverse solution by passing it back through forward kinematics.

Trajectory and safety behavior

- Never jump directly between distant joint commands.
- Use timed waypoints or the ROS 2 Joint Trajectory Controller.
- Enforce position, velocity, and acceleration limits in more than one layer where practical.
- Stop or hold position after stale commands.
- Keep the gripper and payload away from the table during base rotation.
- Test every new sequence in simulation before running it on hardware.

5. Validation Checklist

Mechanical

- No interference throughout the software-limited joint range.
- Bearings support external loads rather than relying solely on actuator shafts.
- Fasteners remain accessible and do not loosen during repeated motion.
- Cables remain clear of joints and do not limit motion.
- The arm holds the intended payload without excessive deflection or overheating.

ROS and software

- The URDF model matches the physical link lengths and joint axes.
- All joint states have correct names, signs, units, and timestamps.
- The TF tree is complete and free of disconnected frames.
- Forward kinematics agrees with the TF end-effector position.
- Inverse kinematics rejects unreachable targets cleanly.
- Simulation and physical hardware use the same high-level command interface.
- Communication loss produces a safe response.

Demonstration acceptance criteria

- The arm homes repeatably.
- The arm reaches at least three predefined locations.
- The gripper picks up the selected lightweight object.
- The arm transfers the object to one of two bins.
- The arm returns to home without collision.
- At least eight of ten repeated cycles complete successfully.
- The project can be launched and operated using documented commands.

6. Scope Protection

Do not include in the initial version

- Computer vision or object detection.
- A rotating or multi-axis wrist.
- Force or torque control.
- Custom circuit-board design.
- High payloads, long reach, or high-speed motion.
- Complex gearboxes unless they are the main SolidWorks learning objective.
- Autonomous grasp planning for objects in arbitrary orientations.

Optional stretch goals after the core system works

- Configure MoveIt for collision-aware motion planning.
- Add a wrist-pitch joint, making the arm 4-DOF plus gripper.
- Port the hardware driver or kinematics node from Python to C++.
- Add a camera and AprilTag-based target localization.
- Add a spring counterbalance and compare actuator effort before and after.
- Create a SolidWorks design table for alternate link lengths or actuators.

7. Definition of Done

The project is complete when the arm exists as a parametric SolidWorks assembly, a valid ROS 2 URDF/Xacro model, a working Gazebo simulation, and a safe physical prototype. It must publish joint states, display correctly in RViz, accept a validated target or named pose, execute a repeatable pick-and-place sequence, and include sufficient documentation for the system to be rebuilt and operated later.